* Assignment No :- 7 *

Q.1] Find First Occurrence :-

Return index of first occurance of target

Input : arr = [5, 2, 3, 2, 4] , target = 2

Output = 1

0	1	2	3	4
5	2	3	2	4

= 5

```
Psvm (){
 int [] arr = { 5, 2, 3, 2, 4 } ;
 int target = 2 ;
 int occur = f (arr , 0 , target) ;
 Print (occur);
}
int f ( int [] arr , int index , target) {

        if ( index == arr length) {
                return -1 ;
        }

        if ( arr [index] == target) {
                return index ;
        }

        return f( arr , index+1 , target);
}
```

Dry Run

→ next page

0	1	2	3	4
5	2	3	2	4

```
Psvm(){
int[]arr = {5,2,3,2,4}
int target = 2;
int occur = f(arr, 0, target);
Print(occur);
}

int f(int[]arr, int index; int target){
        if (index == arr.length){
                return -1;
        }
        if (arr[index] == target){
                return index;
        }
        return f(arr, index+1, target);
}

intf(int[]arr, int index, int target){
        if (index == arr.length){
                return -1;
        }
        if (arr[index] == target){
                return index;
        }
}
}
```

f(0)
f(1)

1
1
1
1

~~Proper to~~

Q.5] Find last Occurrance

Return index of last occurrance of target

Input : [5, 2, 3, 2, 4] , target = 2

Output : 3

0	1	2	3	4
5	2	3	2	4

```
Psvm(){
  int arr[] = { 5, 2, 3, 2, 4};
  int target = 2;
  int occur = f(arr, ~~arr.length~~ 0 ; target);
   print(occur);
}
int f ( int [] arr, int index 0 ; int target) {
        if (index = = arr.length){
            return -1;
        }
        int right = f(arr, index+1 1 ; target);
        if (right != -1){
            return right;
        }
        if (arr[index] = = target){
            return index;
        }
        return -1;
}
int f ( int[]arr, int index 1 ; int target){
        if (index = = arr.length){
            return -1;
        }
        int right = f(arr, index+1 ; target);
        if (right != -1){
            return right;
        }
        if (arr[index] = = target){
            return index;
        }
        return -1;
}
```

```
int f (int[] arr, int index, int target) {
    if (index == arr.length) {
        return -1;
    }
    int right = f (arr, index+1, target);
    if (right != -1) {
        return right;
    }
    if (arr[index] == target) {
        return index;
    }
    return -1;
}

int f (int[] arr, int index, int target) {
    if (index == arr.length) {
        return -1;
    }
    int right = f (arr, index+1, target);
    if (right != -1) {
        return right;
    }
    if (arr[index] == target) {
        return index;
    }
    return -1;
}

int f (int[] arr, int index, int target) {
    if (index == arr.length
        return -1;
    }
    int right = f (arr, index+1, target);
    if (right != -1) {
        return right;
```

```
if (arr[index] == target){
        return index;
}
return -1
}

int f (int[] arr, int index, int target){
    if (index == arr.length){
        return -1;
    }
```

-1

Q.3] Count occurences

Count how many times target appear

Input [1, 2, 2, 3, 2], target = 2

Output : 3

0	1	2	3	4
1	2	2	3	2
	①	②		③

```
Prvm (){
int [] arr = {1, 2, 2, 3, 2}
int target = 2;
int count = 0;
int ccur = f( arr, 0, target, count);
sop ( count);
}

int f (int [] arr, int index, int target, int count){
        if (index == arr.length){
            return -1;
        }
        if (arr[index] == target){
            count ++;
        }
        return f( arr, index+1, target, count);
}
```

Q3]

```
Prum () {

int [] arr = { 1, 2, 2, 3, 1, 2 }
int target = 2;     int count = 0;
int count = f (arr, 0, target, 0)
sop (count);

}
int f ( int [] arr, int index, int target, int count) {
        if ( index = = arr.length) {
                return count;
        }
        if (arr[index] = = target) {
                count++;
        }
        return (arr, index+1 ; target, count);
}
```

Q.4] Check if Array is sorted
Return true if sorted in ascending order.

Input : [1, 2, 3, 4];
Output : true

0	1	2	3
1	2	3	4

```
Prvm(){ int target = arr[0]
int [] arr = { 1, 2, 3, 4 };
boolean b = ~~false~~ true;
int sort = f( ~~int[]~~ arr, ~~boolean~~ b );
```

1 > 2 ×

```
int f (int[] arr, int index , boolean b) {
        if (index == arr length]){
                return -1;
        }
        ~~for(int i = index+1; i < arr.length ; i++){~~
                ~~if ( target > arr[i]){~~
                        ~~b = false;~~

        if ( target > arr[index+1] {
                b = false
                return;
        }
        target =
```

1 >=
1 < 2
2 < 3
3 < 4

~~int f~~

```
Pavm(){
    int [] arr = {1,2,3,4};
    boolean sort = f(arr,0);
        sop (sort);
}
```

0	1	2	3	4
1	2	3	4	5

4

```
boolean f ( int [] arr, int index){
        if ( index = = arr.length -1){
            return true;
        }

        for (int i = 0, i<
        if (arr[index] > arr[index+1] {
            return false;
        }

        return f(arr, index+1);
}
```

0 - 0

0 - 1 > 2 ×

1 - 2 > 3 ×

2 - 3 > 4 ×

3 3 4 > 5 ×

Q.5] Check if All Elements are Even

Return true if all elements are even.

Input = [2,4,6]
Output = true;

0	1	2
2	4	6

```
psvm () {
  int [] arr = { 2,4,6 };

boolean~~int~~ even = f ( arr, 0 );
      sop (even);
}

boolean ~~int~~ f ( int arr[], int index ) {
        if (index = = arr.length) {
              return true;
        }

        if (arr[index] % 2 != 0) {
              return false;
        }
        return f(arr, index+1);
}
```

2%2 = 0
4%2 = 0
6%2 = 0

Q8] Check if Any Element is Negative

Return true if at least one negative exists

Input = [1, 2, -3, 4]
Output - true

0	1	2	3
1	2	-3	4

```
Prvm (){
    int [] arr = { 1, 2, -3, 4 }

boolean negative = f (arr, 0);
    Sop (negative);
}
```

1 < 0
2 < 0
-3 < 0 → true

```
boolean f (int arr[], int index){
        if (index == arr.length){
            return false;
        }
        if (arr [index] < 0){
            return true;
        }
    return f (arr, index+1);
}
```

Q.1] Find maximum Element
Return maximum value in array

Input : [3, 9, 1, 7]
Output : 9

```
Psvm () {

int arr[] = { 3, 9, 1, 7 };

int max = f (arr, 0, 0);
Sop (max);

}

int f ( int[] arr, int index, int max) {
        if ( index = = arr.length) {
                return max;
        }

        if ( max < arr[index]) {
                max = arr[index];
        }

    return f ( arr, index+1, max);

}
```

0 < 3
max = 3
3 < 9
max = 9
9 < 1 ✗
9 < 7 ✗

Q8] Find Minimum Element

Return minimum value;

Input: [3, 9, 1, 7]

0	1	2	3
3	9	1	7

3 < 9
3

3 < 9
3

9 < 1

8 > 9

3 > 1
1

1 > 7

```
prvm (){
    int arr = { 3, 9, 1, 7};

    int min = f (arr, 1, arr[0]);
    Sop(min);

}

int f (int[]arr, int index, int min) {
    if (index == arr.length){
        return min;
    }
    if ( min > arr[index]){
        min = arr[index];
    }

    return f (arr, index+1, min);

}
```

Q.9] Sum of Elements at Odd index

Return sum of elements at index 1, 3 ...

Input [10, 20, 30, 40, 50]

output = 20 + 40 = 60.

0	1	2	3	4
10	20	30	40	50

```
Psvm () {
    int[] arr = {10, 20, 30, 40, 50}

    int sum = f(arr, 0, 0);
    sop(sum);
}

int f( int[] arr, int index, int sum){
        if (index == arr length){
            return sum;
        }

        if (index % 2 != 0){
            sum = sum + arr[indx];
        }

        return f(arr, index+1, sum);
}
```

Q.10] Check palindrome Array

Return true if array is palindrome.

Input : [1, 2, 3, 2, 1]
Output = true

0	1	2	3	4
1	2	3	2	1

→

4	3	2	1	0
1	2	3	2	1

```
Psvm (){
  int [] arr = { 1, 2, 3, 2, 1 }
  ~~int n = arr.length-1~~
boolean ~~int~~ palindrome ~~= f(arr, 0)~~ , = f( arr, 0, arr.length-1)
  Sop (palindrome);
}
boolean f ( int [] arr, int index, ~~int n~~) {
        if (index == arr.length) {
                return true;
        }
        if ( arr[index] != arr[~~arr.length-1~~ palindrome]
                return false;
        }
        palindrome ~~--~~;

        return f( arr, index+1, palindrome);
```

A+B = A bwa ...

Q.13] Count Elements Greater Than X

Q.11] Find second maximum

Return second largest element

Input : [10, 5, 8, 20, 15]

Output : 15

0	1	2	3	4
10	5	8	20	15

→

0	1	2	3	4
20	15	10	8	5

arr[i] < arr[index]

if arr[i] = arr[index+1]

10 < 5 × 0

5 < 8 ②

10 8 5 20 15

5 < 20 ②

10 8 20 5 15

5 < 15 ①

10 8 20 15 5

10 < 5 (20)

10 < 8

10 < 20

20 < 15

20 5 8 10 15

A+B - A bracon cpy

Q 13] Count Elements Greater Than X

Return count of elements greater than given value.

Input : ~~[1, 2, 3, 3]~~ [1, 5, 3, 7, 2] , x = 3
Output : 2

```
Psvm() {
  int []arr = {1, 5, 3, 7, 2}
  int target = 3;
  int count = f(arr, 0, 0, target);
  sop(count);

}
int f( int []arr, int index, int count, int target) {
        if (index == arr length) {
            return count;
        }
        if ( arr[index] > target) {
            count ++;
        }
        return ( arr, index+1, count, target);
}
```

1 > 3 × 1 > 3 × 2 > 3 ×
5 > 3 ✓ 7 > 3 ✓

Q.14] Check if Array Contains Target

Return true if target exists

Input: [1, 2, 3, 4] , target = 3
Output: true;

0	1	2	3
1	2	3	4

1 == 3 ✗
2 == 3
3 == 3 ✓

```
Psvm () {
  int [] arr = { 1, 2, 3, 4 };
  int target = 3;

  boolean Contain = f ( arr, 0 , target) ;
    sop (contain);
}
boolean f ( int [] arr, int index, int target) {
        if (index == arr.length) {
            return false;
        }

        if (arr[index] == target) {
            return true;
        }

        return f(arr, index+1, target);
}
```

Q.15] Find sum of All Negative Numbers

Return sum of all negative elements.

Input = [1, -2, 3, -4, 5]
Output : -6.

less than 0 → add.

```
Psvm() {
    int[]arr = { 1, -2, 3, -4, 5};

    int sum = f ( arr, 0, 0);
    sop(sum);

}
int f ( int[] arr, int index, int sum) {
        if ( index == arr.length {
            return sum;
        }

        if (arr[index] < 0) {
            sum = sum + arr[index];
        }

        return ( arr, index+1, sum);
}
```

Q.16] Find Index of smallest element

Return index of minimum element

Input = [5, 2, 8, 1, 3]
Output = 3

0	1	2	3	4
5	2	8	1	3

5>2
min=2
2>8
2>1
min=1
1>3

```
Psum(){
  int[]arr={5,2,8,1,3}
                    arr[0]
  int index = f(arr, 0, ↓ );
  sop(index);
}
int f ( int[]arr, int index, int min){
        if ( index == arr.length){
                return -1;
        }
        if (min > arr[index]){
                min = arr[index];
                return index;
        }
        return f(arr, index+1, min);
}
```

Q17] Count Elements Divisible by K

Return count of elements divisible by k.

Input : [2, 4, 6, 6, 9] , k = 2

Output = 3

0	1	2	3	4
2	4	5	6	9

2 % 2 = 0 ① 5 % 2 = 1 ×

4 % 2 = 0 ② 6 % 2 = 0 ③

9 % 2 = 1 ×

```
Psvm() {

int arr[] = { 2, 4, 5, 6, 9 }
int k = 2;

int count = f( arr, 0, k, 0);
 sop (count);

}

int f ( int[] arr, int index, int k , int count){
        if (index = = arr.length) {
                return count;
        }

        if (arr[index] % k = = 0){
                count ++;
        }
        return (arr, index+1, k , count);

}
```

Q:18) Check if All Elements are Unique

Return true if no duplicates exist

Input : [1, 2, 3, 4]
Output : true

0	1	2	3
1	2	3	4

1 != 2 ✓
2 != 3 ✓
3 != 4 ✓

4 - 1 = 3

```
Psvm () {
    int [] arr = { 1, 2, 3, 4 }

    boolean unique = f (arr, 0);
    sop (unique);
}

boolean f ( int [] arr, int index ) {

        if (index == arr length - 1) {
                return false;
        }
        if ( arr[index] != arr[index+1]) {
                return true;
        }

        return ( arr, index+1 );
}
```

19] Find sum of Digits of All Elements

Each element is a number → sum all digits recursively.

Input : [12 , 34, 5]
Output : 1+2+3+4+5 = 15

12

```
{ while ( arr[index] num > 0) {
    int rem = arr[index] num % 10
      sum = sum+rem;
    arr[index] num = arr[index] num / 10;
  }
```

```
Psvm () {
    int []arr = { 12, 34, 5 };

    int sum = f( arr, 0, 0 );
    Sop(sum);
}
int f (int []arr, int index , int sum) {
        if ( index == arr.length) {
            return sum;
        }

        while (arr[index] > 0) {
            int rem = arr[index] % 10;
              sum = sum+rem;
        arr[index] = arr[index] / 10;
        }
        return f (arr, index+1 , sum);
}
```